

CURSO DE ENGENHARIA DE SOFTWARE

Disciplina de Laboratório de Programação

Aula 09: Recursividade em C

Prof. Me. Ivandro José de Freitas Rocha

Goianésia – 2026

O estudante saberá definir recursão (caso-base e passo recursivo), implementar funções recursivas em C (fatorial, Fibonacci e quicksort), explicar a pilha de execução e comparar abordagens recursivas e iterativas.

CONTEÚDOS

- Fundamentos da recursão: revisão de funções, definição circular, caso-base e passo recursivo;
- Função fatorial: expansão do cálculo, versões iterativa e recursiva;
- Sequência de Fibonacci: implementação recursiva e iterativa, análise de eficiência;
- Algoritmo quicksort: particionamento, divisão e conquista, implementação recursiva e análise do programa;
- Síntese e prática: recursão versus iteração, erros comuns e exercícios propostos.

REFERÊNCIAS

- DEITEL & DEITEL. C: como programar. 6. ed. (Cap. 8)
- SCHILDT. C completo e total. 3. ed. (Cap. 5)

FUNÇÕES PODEM CHAMAR OUTRAS FUNÇÕES

Em C, funções são blocos de código reutilizáveis que podem invocar outras funções.

A função `main()`, ponto de entrada do programa, pode chamar funções da biblioteca padrão, como `printf()` ou funções definidas pelo programador.

Quando uma função passa a invocar a si mesma, caracteriza-se o que denominamos **função recursiva**.

Recursão, também denominada **definição circular**, ocorre quando um conceito é definido em termos de si mesmo.

Na programação, uma função é recursiva quando, para resolver um problema, invoca a si própria com uma instância menor do mesmo problema.

O fatorial ilustra a ideia: $3! = 3 \times 2!$, $4! = 4 \times 3!$ e, de modo geral, $n! = n \times (n - 1)!$.

CASO-BASE E PASSO RECURSIVO

A estrutura fundamental de toda recursão

Todo processo recursivo assenta-se sobre dois pilares:

- o caso-base, cuja solução é imediata e conhecida.
- o passo recursivo, método geral que reduz o problema original a um ou mais subproblemas de mesma natureza.

Identificar essas duas partes é condição necessária para definir qualquer função recursivamente.

EXEMPLO: o que ocorre no fatorial e no n -ésimo termo da sequência de Fibonacci.

DEFINIÇÃO MATEMÁTICA

O fatorial de um inteiro não negativo n pode ser definido recursivamente:

$$n! = \begin{cases} 1, & \text{se } n = 0 \text{ (**caso – base**)} \\ n * (n - 1)!, & \text{se } n > 0 \text{ (**caso recursivo**)} \end{cases}$$

Essa definição estabelece um processo recursivo:

- para calcular $n!$, basta calcular $(n - 1)!$ e
- multiplicar o resultado por n .

DESMEMBRANDO 4! PASSO A PASSO

A expansão do cálculo explicita a cadeia de dependências:

$$0! = 1 \text{ (**caso-base**)}$$

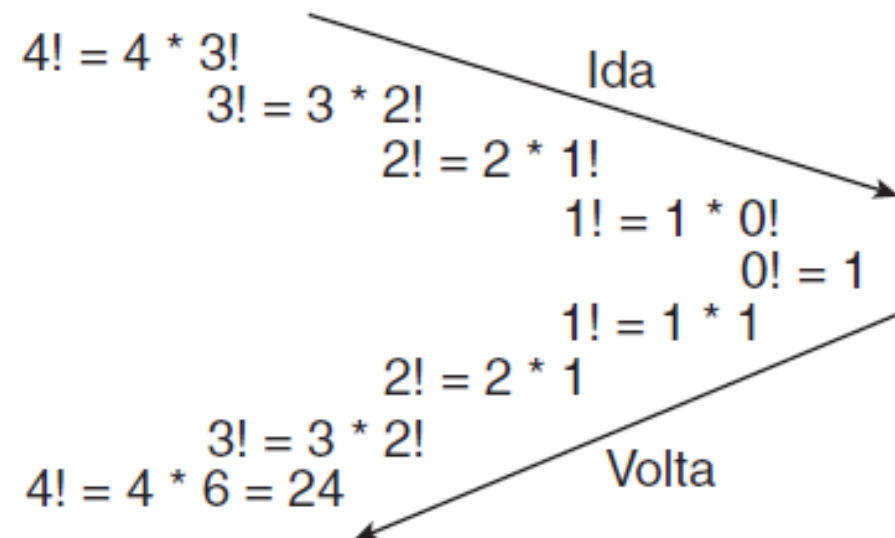
$$1! = 1 * 0!$$

$$2! = 2 * 1!$$

$$3! = 3 * 2!$$

$$4! = 4 * 3!$$

$$n! = n * (n - 1)! \text{ (**fórmula geral**)}$$



Cada termo depende do anterior, até alcançar o caso-base, a **fórmula geral** $n! = n * (n - 1)!$ conduz, inevitavelmente, a $0! = 1$.

COMPARATIVO ENTRE OS MÉTODOS

Duas implementações, um mesmo resultado

O mesmo problema admite duas abordagens: a **iterativa**, baseada em laços de repetição, e a **recursiva**, baseada na autoinvocação da função. Ambas produzem resultados idênticos, porém com estruturas de controle distintas.

// exemplo com iteratividade

```
int fatorial(int n){  
    if(n == 0) return 1;  
    else{  
        int i; int f=1;  
        for(i=2; i<=n; i++) f=f*i;  
        return f;  
    }  
}
```

// exemplo com recursividade

```
int fatorial(int n){  
    if(n == 0) return 1;  
    else  
        return n*fatorial(n-1);  
}
```

QUANDO A RECURSÃO DEVE PARAR

Critério de parada e progressão do parâmetro

Formulações recursivas costumam ser mais concisas e elegantes que as iterativas. Em contrapartida, exigem mais memória, pois cada chamada ocupa espaço na pilha de execução.

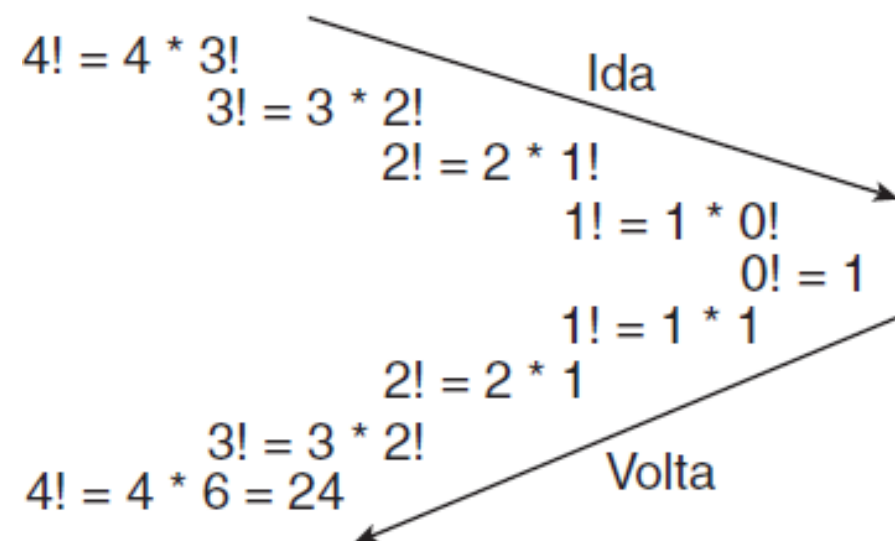
Duas condições garantem a terminação de uma função recursiva:

- a existência de um **critério de parada**, que determina quando cessam as autoinvocações, e
- a **modificação do parâmetro** a cada chamada, assegurando que a recursão progrida em direção ao caso-base.

SIMULANDO A CHAMADA NA PRÁTICA

O cálculo de $4!$ pode ser expandido passo a passo:

$$\begin{aligned}4! &= 4 * 3! \\ &= 4 * (3 * 2!) \\ &= 4 * (3 * (2 * 1!)) \\ &= 4 * (3 * (2 * (1 * 0!))) \\ &= 4 * (3 * (2 * (1 * 1))) \\ &= 4 * (3 * (2 * 1)) \\ &= 4 * (3 * 2) \\ &= 4 * 6 \\ &= 24\end{aligned}$$



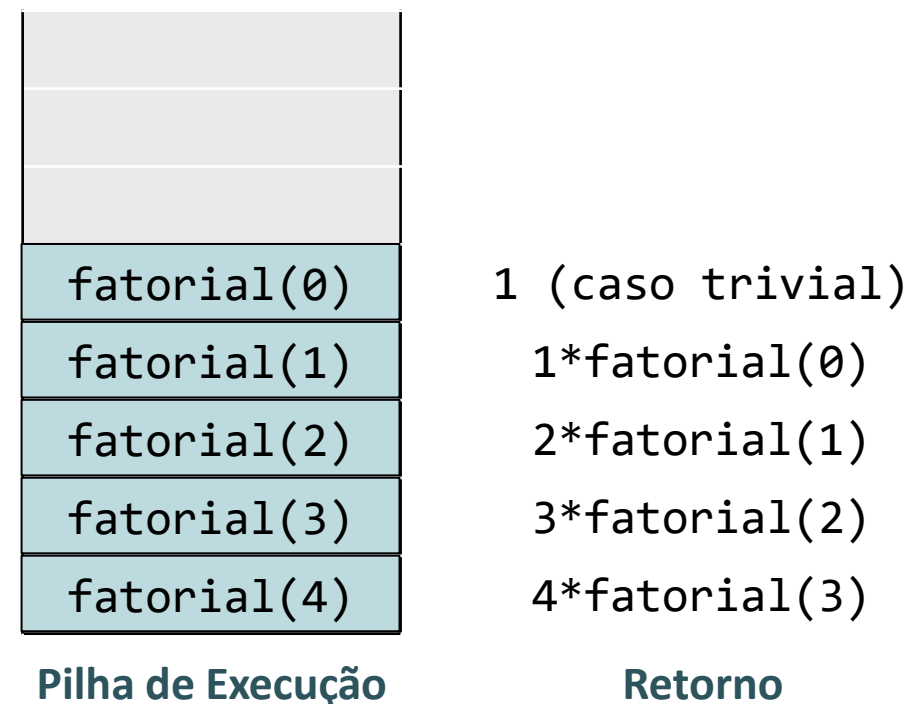
Subjacente a esse processo está o mecanismo da **pilha de execução**, estrutura de dados que o computador utiliza para controlar as chamadas de função.

PILHA DE EXECUÇÃO

Empilhamento

A pilha de execução organiza as chamadas recursivas: cada invocação de `fatorial()` empilha um novo quadro contendo seus parâmetros e endereço de retorno.

```
int fatorial(int n){  
    if(n == 0) //critério de parada  
        return 1;  
    else  
        //parâmetro de fatorial sempre muda  
        return n*fatorial(n-1);  
}
```

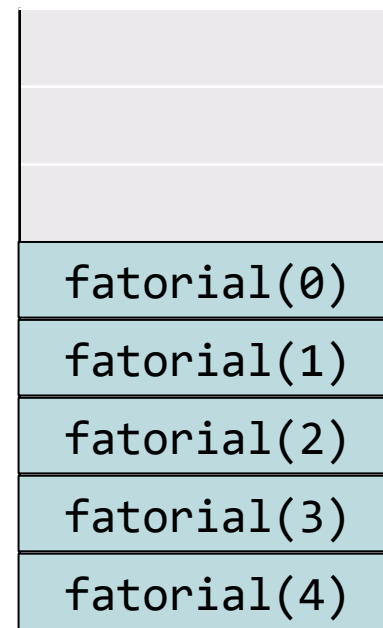


PILHA DE EXECUÇÃO

Desempilhamento

Ao atingir fatorial(0), o caso-base retorna 1 e inicia-se o desempilhamento, no qual cada quadro retoma o cálculo interrompido:

```
int fatorial(int n){  
    if(n == 0) //critério de parada  
        return 1;  
    else  
        //parâmetro de fatorial sempre muda  
        return n*fatorial(n-1);  
}
```



Pilha de Execução

1 (caso trivial)

1*1

2*1*1

3*2*1*1

4*3*2*1*1

Retorno

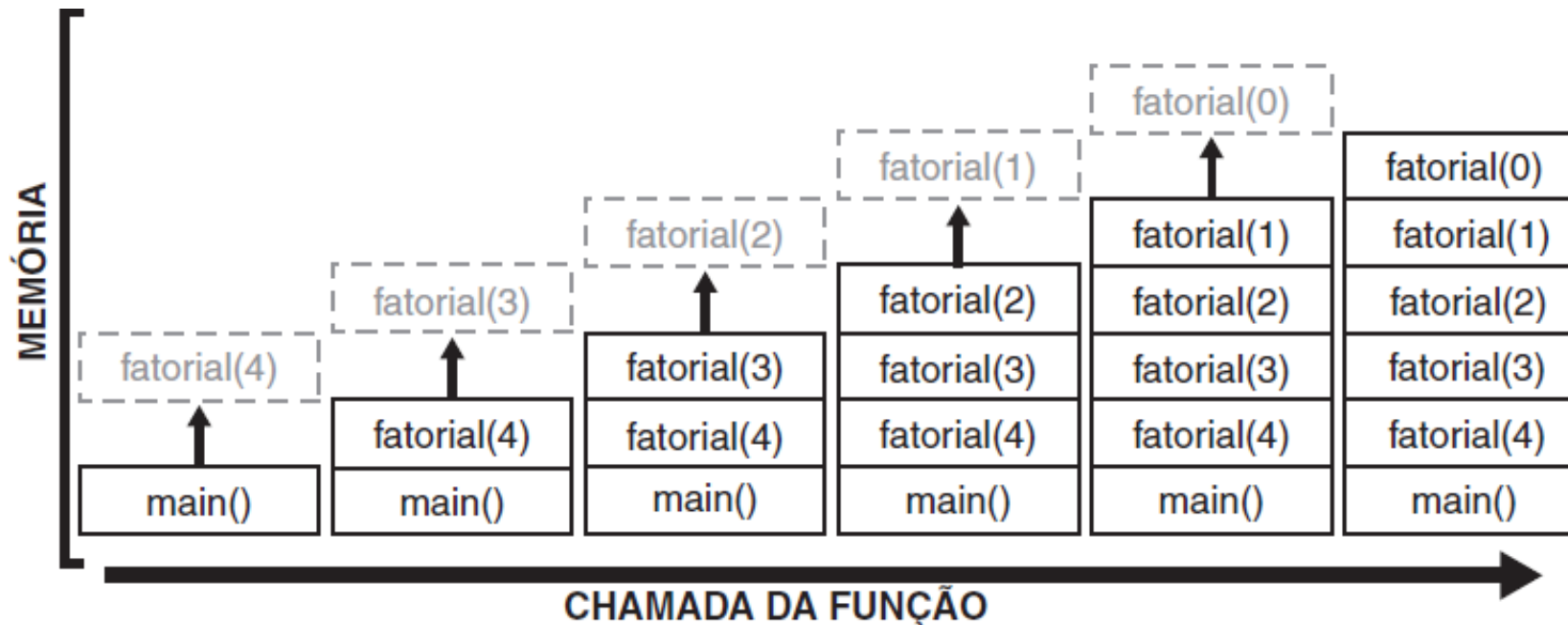
SIMULANDO A CHAMADA NA PRÁTICA

```
int fatorial(int n){  
    if(n == 0) //critério de parada  
        return 1;  
    else  
        //parâmetro de fatorial sempre muda  
        return n*fatorial(n-1);  
}
```

Executa a instrução `int x = fatorial(4)`.

A função é invocada com $n = 4$ e, como $n \neq 0$, avalia a expressão $4 * \text{fatorial}(3)$, o que exige uma nova invocação, com $n = 3$.

Esse encadeamento de chamadas prossegue até que o caso-base seja alcançado.

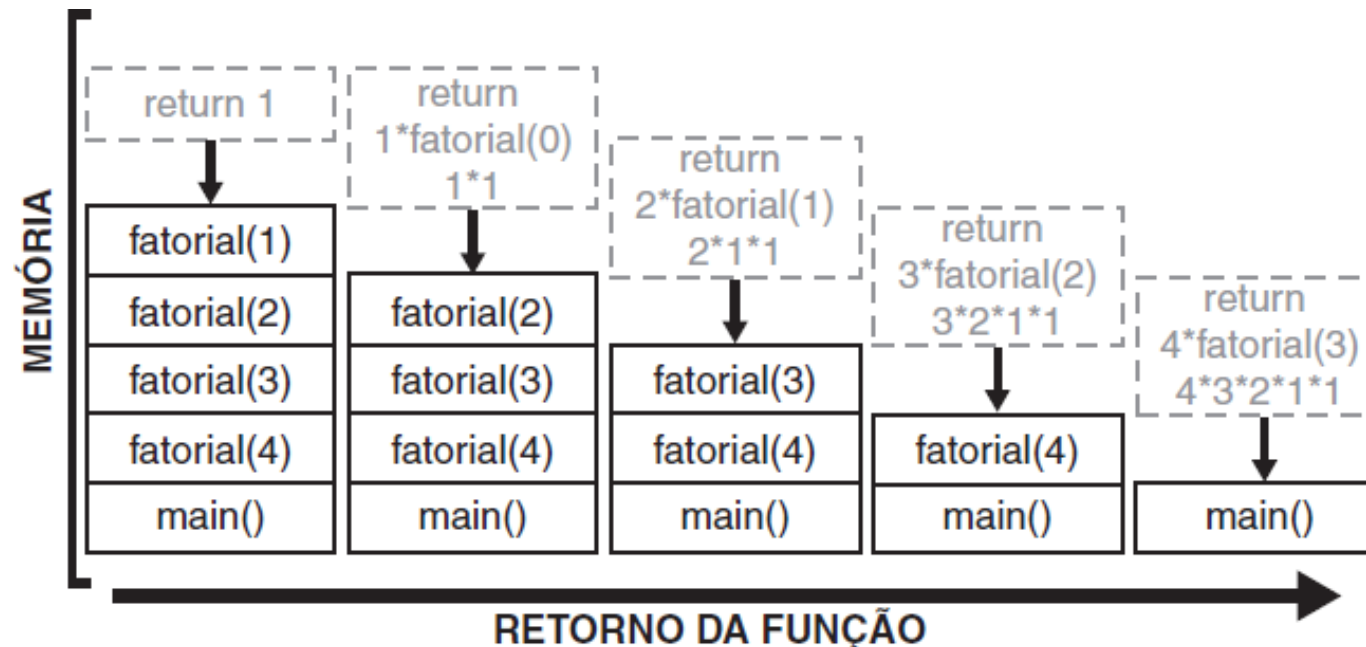


SIMULANDO A CHAMADA NA PRÁTICA

```
int fatorial(int n){  
    if(n == 0) //critério de parada  
        return 1;  
    else  
        //parâmetro de fatorial sempre muda  
        return n*fatorial(n-1);  
}
```

Ao alcançar o caso-base, a recursão inverte seu curso: cada chamada pendente retoma sua execução, retornando o valor calculado à chamada anterior.

Esse processo de retorno é denominado desempilhamento.



CASO-BASE E PASSO RECURSIVO

A sequência de Fibonacci é definida por: $fib(0) = 0$, $fib(1) = 1$ e, para $n > 1$, $fib(n) = fib(n - 1) + fib(n - 2)$.

Em outras palavras, cada termo, a partir do segundo, é a soma dos dois termos imediatamente anteriores: 0, 1, 1, 2, 3, 5, 8, 13, ...

Sendo assim, o n-ésimo número $fib(n)$ é dado por:

$$fib(n) = \begin{cases} 0, & \text{se } n = 0 \\ 1, & \text{se } n = 1 \\ fib(n - 2) + fib(n - 1), & \text{se } n > 2 \end{cases}$$

FUNÇÃO RECURSIVA

A sequência de Fibonacci é um exemplo clássico de problema recursivo.

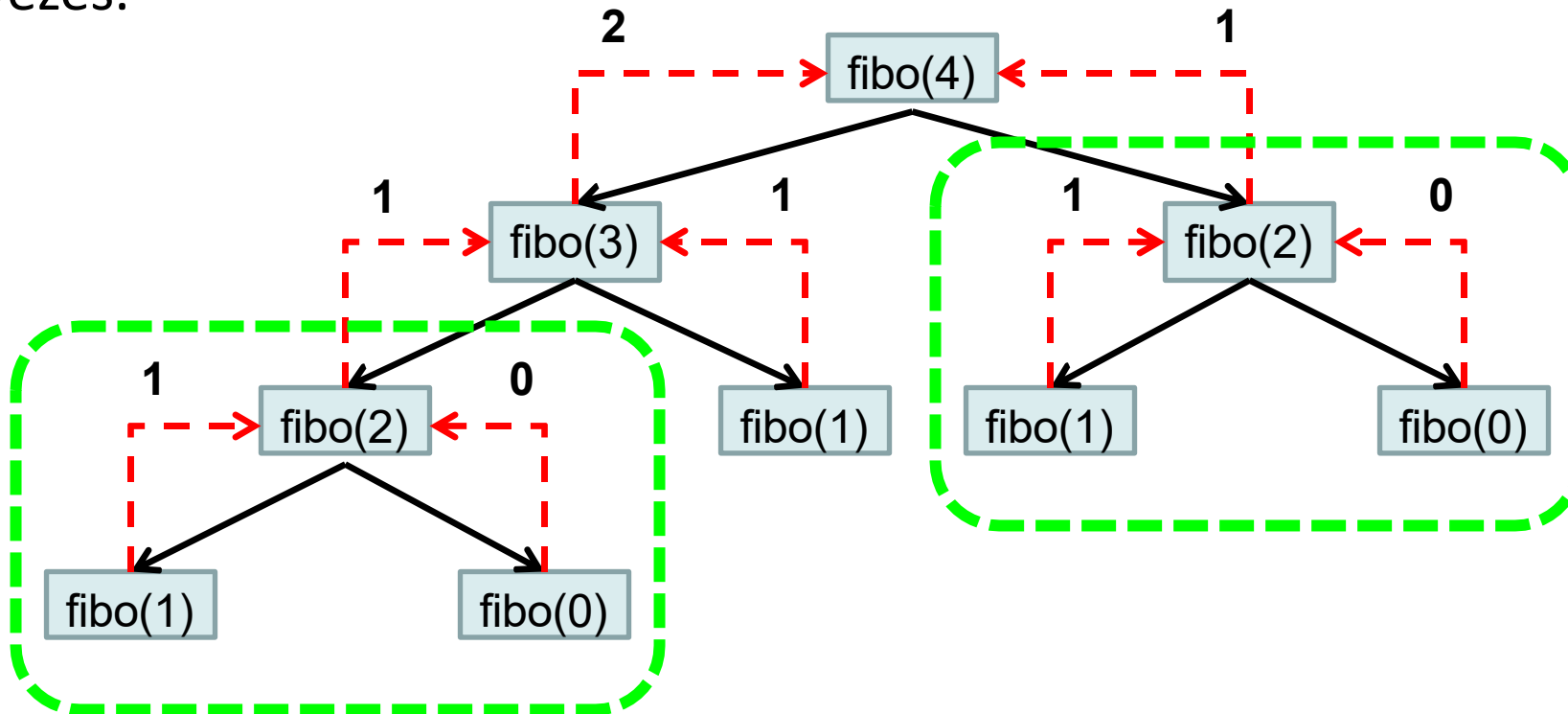
Sua solução recursiva é notavelmente concisa: o **caso-base** cobre $n = 0$ e $n = 1$, e o **passo recursivo** decompõe $\text{fibo}(n)$ em $\text{fibo}(n - 1) + \text{fibo}(n - 2)$.

```
int fibo(int n){  
    if (n == 0 || n == 1)  
        return n;  
    else  
        return fibo(n-1) + fibo(n-2);  
}
```

SIMULANDO A CHAMADA NA PRÁTICA

A árvore de chamadas de `fibonacci(4)` revela o custo da abordagem: `fibonacci(2)` é calculado duas vezes e `fibonacci(1)`, três vezes.

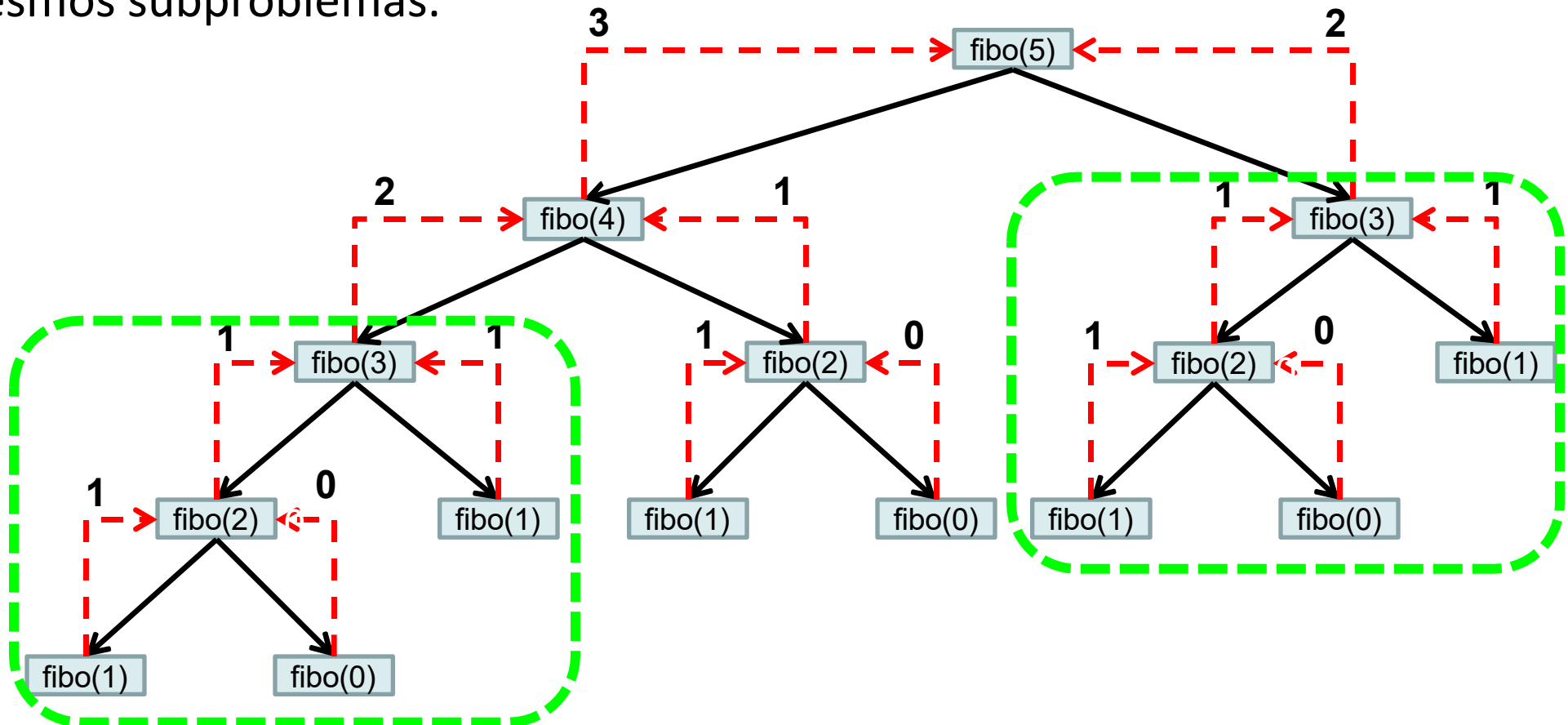
A elegância do código não implica eficiência, os mesmos valores são recalculados inúmeras vezes.



SIMULANDO A CHAMADA NA PRÁTICA

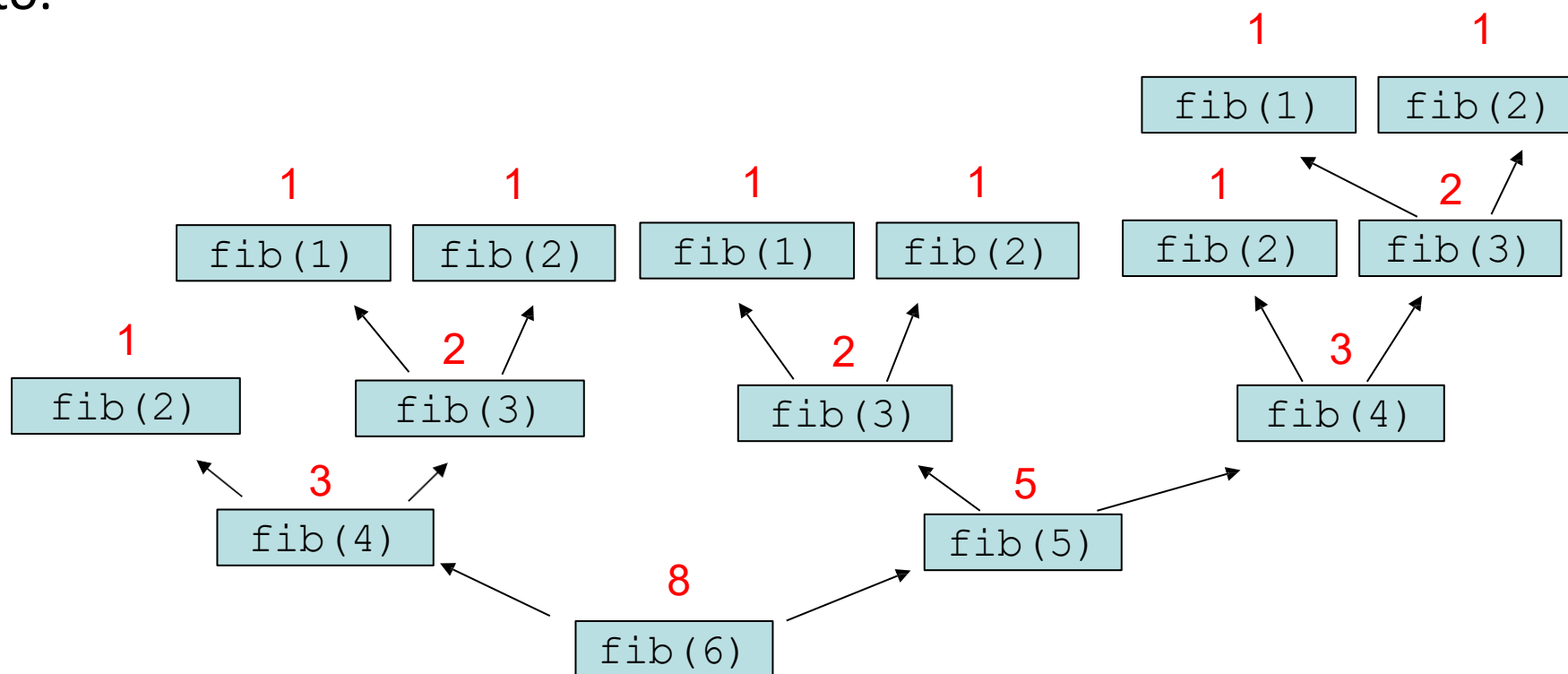
Ao elevar o argumento para `fibonacci(5)`, a árvore de chamadas cresce de forma acentuada.

Esse crescimento evidencia que a versão recursiva ingênua recalcula repetidamente os mesmos subproblemas.



SIMULANDO A CHAMADA NA PRÁTICA

A árvore de chamadas de `fib(6)` ilustra a redundância computacional: subárvores idênticas são recalculadas de forma independente, multiplicando o esforço total de processamento.



FUNÇÃO ITERATIVA

A **versão recursiva** é extremamente ineficiente: recalcula os mesmos valores inúmeras vezes.

A **versão iterativa** elimina essa redundância ao armazenar apenas os dois últimos termos:

```
int fib(int n){  
    int a = 0, b = 1, c, i;  
    if(n == 0) return a;  
    for(i = 2; i <= n; i++){  
        c = a + b;  
        a = b;  
        b = c;  
    }  
    return b;  
}
```

COMPARATIVO DE TEMPOS DE EXECUÇÃO

Brassard e Bratley (Fundamentals of Algorithmics, 1996, p. 73) comparam os tempos de execução das versões recursiva e iterativa de Fibonacci: para **n = 100**, a versão recursiva demandaria cerca de **109 anos**, contra **1,5 ms** da iterativa.

n	10	20	30	50	100
recursivo	8 ms	1 s	2 min	21 dias	10 ⁹ anos
iterativo	0.17 ms	0.33 ms	0.50 ms	0.75 ms	1,50 ms

CONCLUSÃO: um algoritmo recursivo nem sempre é a melhor solução; contudo, a recursividade frequentemente confere maior simplicidade e clareza ao algoritmo.

ORDENAR UM VETOR EM ORDEM CRESCENTE

O *quicksort* é um algoritmo de ordenação que classifica os elementos de um vetor em ordem crescente.

Sua estratégia consiste em:

- escolher um **pivô m**, posicionado no centro do vetor;
- percorrer o vetor com dois índices (**i**, a partir do primeiro elemento, e **j**, a partir do último);
- enquanto $i \leq j$, incrementar **i** até encontrar um elemento maior que o pivô, decrementar **j** até encontrar um elemento menor que o pivô e, então, trocar os elementos nas posições **i** e **j**.

DIVIDIR PARA CONQUISTAR

Ao final do particionamento, o vetor apresenta o seguinte invariante:

- à esquerda do pivô, todos os elementos são menores ou iguais a m ;
- à direita, todos são maiores ou iguais a m .

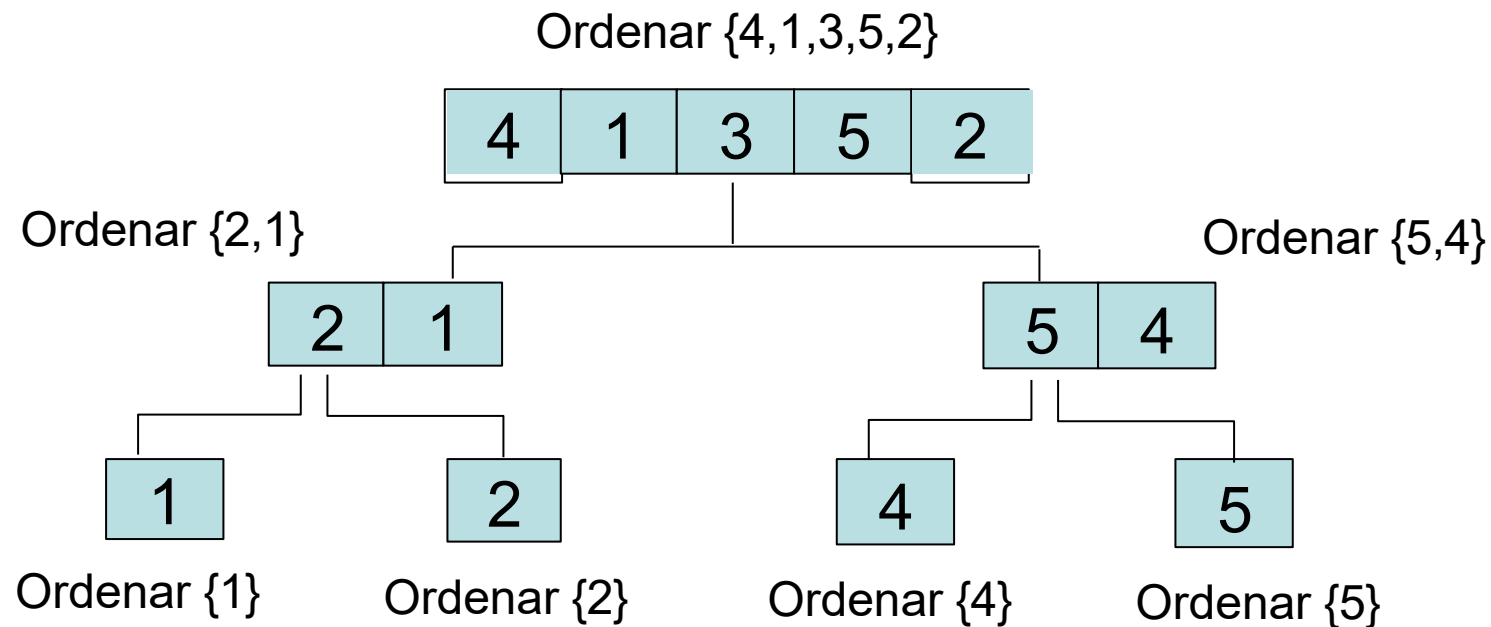
O problema original reduz-se, então, a ordenar cada uma das metades.

O mesmo procedimento é aplicado recursivamente a cada nova metade, até que cada uma contenha um único elemento (o **caso-base** da recursão).

PARTICIONAMENTO E CHAMADAS RECURSIVAS

Considere o vetor {4, 1, 3, 5, 2}. Após o particionamento, cada metade é ordenada recursivamente com o mesmo método.

Como a natureza de cada subproblema é idêntica à do problema original (ordenar um vetor), o mesmo algoritmo pode ser aplicado a cada subproblema, caracterizando a abordagem dividir para conquistar.



IMPLEMENTAÇÃO COMPLETA E ANÁLISE DO PROGRAMA

A implementação do quicksort em C reflete a estrutura recursiva discutida: a função particiona o vetor e invoca a si mesma para ordenar cada metade.

```
void quicksort(int v[], int esq, int dir){
    int i=esq, j=dir, m=v[(esq+dir)/2];
    int aux;
    while(i<=j){
        while(v[i]<m) i++;
        while(v[j]>m) j--;
        if(i <= j){
            aux=v[i]; v[i]=v[j]; v[j]=aux;
            i++; j--;
        }
    }
    if(esq<j) quicksort(v, esq, j);
    if(i<dir) quicksort(v, i, dir);
}
```

A função quicksort é recursiva: após particionar o vetor, invoca a si mesma para ordenar cada metade.

A recursividade é uma estratégia poderosa para problemas divisíveis em subproblemas de mesma natureza.

O processo termina quando uma metade contém um único elemento (o caso trivial).

QUANDO USAR CADA ABORDAGEM

ITERAÇÃO: repetição com laços (for, while), é mais eficiente em tempo e memória e não corre risco de estouro de pilha.

RECURSÃO: a função chama a si mesma para resolver um problema menor, é mais elegante e natural para problemas divisíveis: árvores, quicksort, Torre de Hanói.

REGRA PRÁTICA: use recursão quando a solução iterativa fica complexa e a profundidade é pequena; use iteração quando desempenho importa.

Critério	Recursão	Iteração
Legibilidade	Código mais enxuto	Pode ficar verboso
Memória	Consome a pilha de execução	Uso constante
Desempenho	Geralmente mais lento	Geralmente mais rápido
Risco	Estouro de pilha	Laço infinito (lógica errada)

RECURSÃO INFINITA E ESTOURO DE PILHA

Esquecer o caso-base: a função nunca para de se chamar (recursão infinita).

```
int fatorial(int n) {  
    return n * fatorial(n - 1); // sem caso-base: recursão infinita!  
}
```

Parâmetro que não muda entre chamadas: a mesma chamada se repete para sempre.

Estouro de pilha (stack overflow): cada chamada ocupa memória na pilha; profundidade excessiva esgota a memória e o programa encerra com erro.

Como evitar:

- Sempre definir o caso-base (critério de parada).
- Garantir que o parâmetro se aproxime do caso-base a cada chamada.
- Testar com valores pequenos antes de valores grandes.

TRÊS NÍVEIS DE DESAFIO

Nível básico

- Soma dos elementos de um vetor (recursiva).
- Máximo Divisor Comum (MDC).
- Contagem regressiva de n até 0.

Nível intermediário

- Inverter uma string recursivamente.
- Busca binária recursiva.
- Potência x^n recursiva.

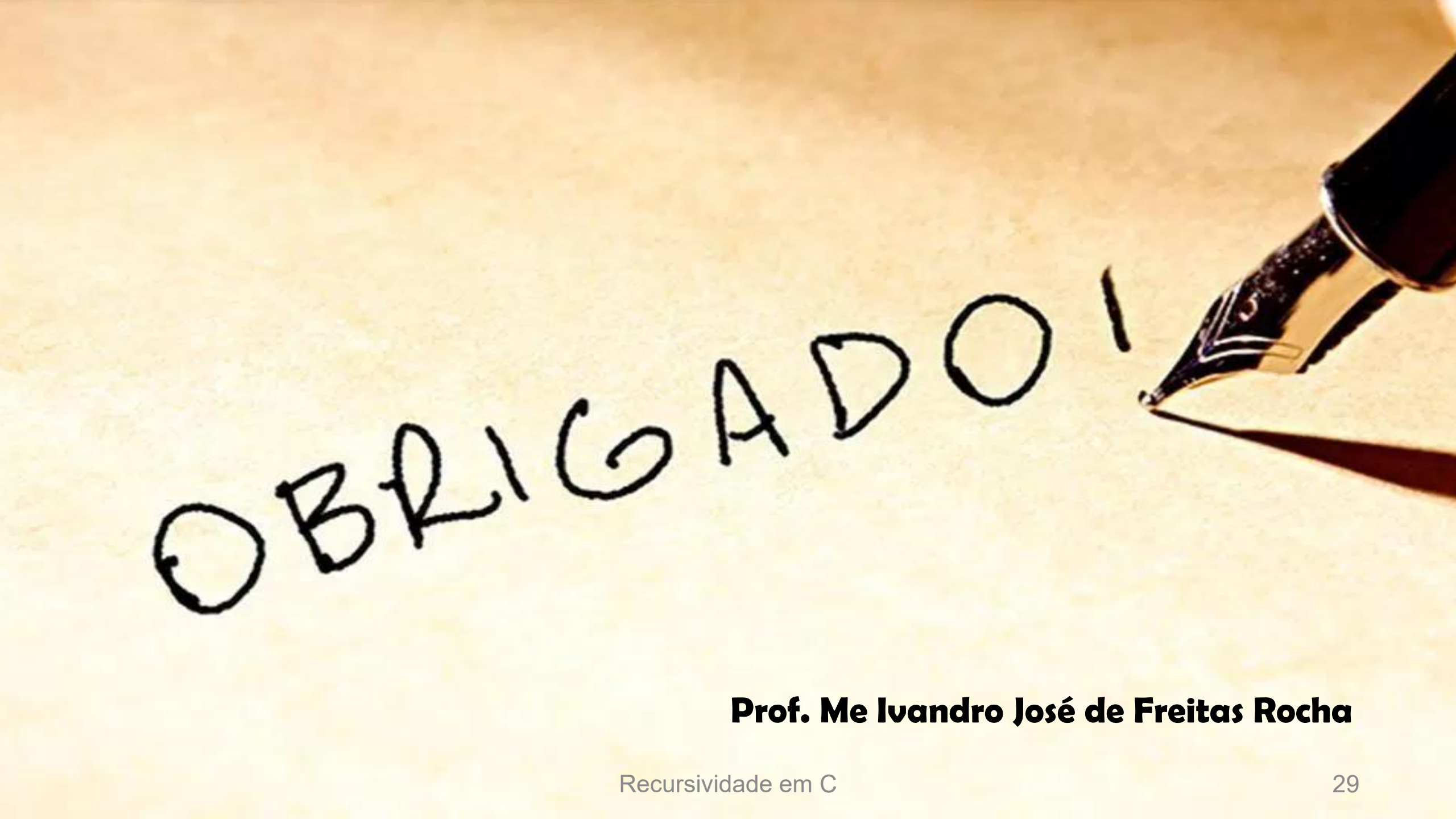
Nível avançado

- Torre de Hanói.
- Gerar todas as permutações de uma string.
- Implementar o quicksort do zero.



ivandro.rocha@uniego.edu.br

DUVIDAS?

A close-up photograph of a fountain pen with a silver-colored nib and a dark barrel, positioned on the right side of the frame. The pen is in the process of writing the word "OBRIGADO!" in a bold, black, handwritten style on a light-colored, textured piece of paper. The letters are slightly slanted and the ink is thick. The exclamation mark is a simple vertical line. The lighting is warm, creating a soft glow on the paper and the pen's nib.

OBRIGADO!

Prof. Me Ivandro José de Freitas Rocha